Preface

Like any of the most-used programming languages, Java has its share of detractors, advocates, issues, quirks,[1] and a learning curve. The *Java Cookbook* aims to help the Java developer get up to speed on some of the most important parts of Java development. I focus on the standard APIs and some third-party APIs, but I don't hesitate to cover language issues as well.

This is the fifth edition of this book, and it has been shaped by many people and by the myriad changes that Java has undergone over its two and a half decades of popularity. Readers interested in Java's history can refer to Appendix A.

Java 21 is the current long-term supported (LTS) version, and Java 24 is the latest current release at the time of publication of this edition. The current cadence of releases every six months may be great for the Java SE development team at Oracle, for developers who want access to the latest-and-greatest, and for click-driven, Java-related news sites, but it "may cause some extra work" for Java book authors, since books typically have a longer revision cycle than Java now does! Java 9, which came out after a previous edition of this book, was a sort-of-breaking release, the first release in a very long time to almost break backward compatibility, primarily the Java Platform Module System.

Everything in this edition of the book can be assumed to work on Java 11 or later, unless specified otherwise (see "Icons"). At this point in time, nobody should be using Java 8 (or anything before it) for anything. Certainly nobody should be doing new development in Java 8. If you are, it's time to move on! While Java 11 is considered an LTS release, it is so old that you shouldn't use it. Java 17 is the previous LTS and quite usable, but if you want the best start on an LTS Java release, begin with Java 21.

The goal of this revision of *Java Cookbook* is to keep the book up to date with all this change. I've included numerous new features, which meant I had to remove a significant amount of material. There are numerous additions throughout the chapters. In several places, a few recipes have been combined into one in the interest of simplicity and readability (and reducing page count). And of course I've updated a lot of other information along the way.

Who This Book Is For

I'm going to assume that you know the basics of Java. I won't tell you how to `println` a string, nor how to write a class that extends another and/or implements an interface. I'll assume you've taken a Java course such as Learning Tree's Introduction (which I wrote the first version of, decades ago) or worked your way through an introductory book such as:

- *Head First Java* by Kathy Sierra and Bert Bates (O'Reilly), a tutorial filled with visual metaphors and other brain-friendly learning techniques
- *Java in a Nutshell* by David Flanagan (O'Reilly), a more concise book covering the standard APIs in more detail
- *Think Java* by Allen Downey and Chris Mayfield (O'Reilly), a hands-on introduction to computer science and programming via Java
- *Learning Java* by Marc Loy, Patrick Niemeyer, and Daniel Leuck (O'Reilly), a book for practicing programmers moving to Java

However, Chapter 1 covers some techniques that you might not know very well and that are necessary to understand some of the later material. Feel free to skip around! Both the printed version of the book and the electronic copy are heavily cross-referenced.

Just to give an idea of how big a topic Java really is, have a look at Figure P-1. An earlier version of this diagram was online at Sun, prior to the Oracle takeover, with each box being a link to the corresponding documentation. Figure P-1 is my re-creation of it, using a simple Java Swing program written for the occasion.

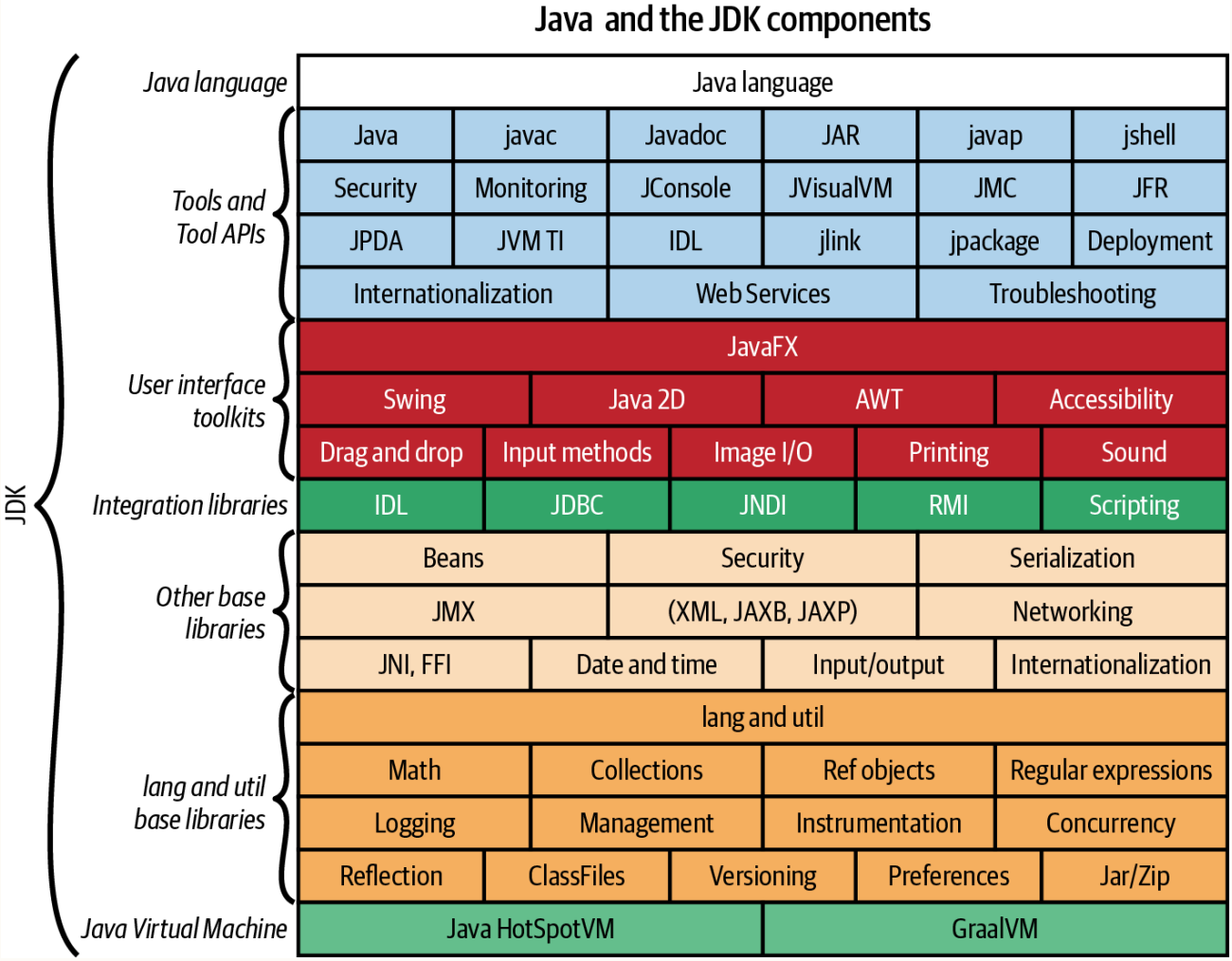


Figure P-1. Java conceptual diagram

What's in This Book?

In the early 2000s, Java seemed better suited to "development in the large," or enterprise application development, than to the few-line, one-off script in Perl, AWK, Python, or the Unix/Linux shells. That's because it is a compiled, object-oriented language. However, Java has gotten better for ad hoc tasks that might previously have been done with a scripting language. JShell (Recipe 1.5) and the run-java-source-file mechanism (see "Discussion") have helped move Java in this direction. I illustrate many techniques with shorter Java class examples and even code fragments; some of the simpler ones will be shown using JShell. All of the code examples over a few lines long are in one of my public GitHub repositories, so you can rest assured that all the code you see here has been compiled and run (most of them run recently).

What's Not in This Book

Since some readers will have used previous editions (my thanks to you!), here's a brief list of information that is no longer included.

The fifth edition removed obsolete recipes from several chapters. The longer program listings at the end of many chapters were removed from the book, replaced by links to view them either from the code repository or online.

The fourth edition removed the discussion of Swing GUI and AWT Graphics, though these are still used in some examples. The code for database access was also removed. The code for the former lives on in the *desktop* project within the *javasrc* repository on GitHub. For the latter, see my *jpademo* GitHub repo.

The removed recipes have been migrated to my website.

Organization of This Book

Let's go over the organization of this book. Each chapter consists of a handful of recipes—short sections that describe a problem and its solution, along with a code example. The code in each recipe is intended to be largely self-contained; feel free to borrow bits and pieces of any of it for use in your own projects. The code is distributed with a Berkeley-style "use with attribution" copyright. Most recipes provide reasonably complete coverage of their topic. Some topics, however, have more detail, more nuance, than can be covered in the recipe format; these end with references that the reader can consult for additional explanation and examples.

I start off Chapter 1, "Getting Started: Compiling and Running Java", by describing some methods of compiling your program on different platforms and running them in different environments (command line, IDE, and windowed desktop). This is followed by Chapter 2, "Software Development, Testing, and Maintenance", which discusses techniques for building real applications. Topics such as Java modules, build tools, unit testing, and continuous integration are discussed here.

The next few chapters deal with basic APIs. Chapter 3, "Strings and Things", concentrates on one of the most basic but powerful data types in Java, showing you how to assemble, dissect, compare, and rearrange "strings," what you might otherwise think of as ordinary text. This chapter also covers the topic of internationalization/localization so that your programs can work as well in Akbar, Afghanistan, Algiers, Amsterdam, and Angleterre as they do in Alberta, Arkansas, and Alabama.

Chapter 4, "String Matching with Regular Expressions", teaches you how to use the powerful regular expressions technology from Unix on strings, in many string-matching

and pattern-matching problem domains. Regex processing has been standard in Java for years, but if you don't know how to use it, you may be reinventing the flat tire.

Chapter 5, "Numbers", deals both with built-in numeric types such as `int` and `double`, as well as the corresponding API classes (`Integer`, `Double`, etc.) and the conversion and testing facilities they offer. There is also brief mention of the "big number" classes.

Because Java programmers often need to deal in dates and times, both locally and internationally, Chapter 6, "Dates and Times", covers this important topic.

The next chapter covers structuring your program's data. As in most languages, arrays in Java are linear, indexed collections of similar objects, as discussed in Chapter 7, "Structuring Data with Java". This chapter goes on to deal with the many Collections classes: powerful ways of storing quantities of objects in the java.util package, including the use of Java Generics.

Despite some syntactic familial resemblance to procedural languages such as C, Java is at heart an object-oriented programming (OOP) language, with some important functional programming (FP) constructs skilfully blended in. Chapter 8, "Object-Oriented Techniques", discusses some of the key notions of OOP as it applies to Java, including the commonly overridden methods of `java.lang.Object` and the important issue of design patterns. Java is not, and never will be, a pure FP language. However, it is possible to use some aspects of FP, increasingly so with Java 8 and its support of lambda expressions (a.k.a. closures). This is discussed in Chapter 9, "Functional Programming Techniques: Functional Interfaces, Streams, and Parallel Collections".

Chapter 10, "Input and Output: Reading, Writing, and Directory Tricks", details the rules for reading and writing files (don't skip this if you think files are boring; you'll need some of this information in later chapters). The chapter also shows you everything else about files—such as finding their size and last-modified time—and about reading and modifying directories, creating temporary files, and renaming files on disk.

Java was one of the first mainstream languages to provide language support for *threads*, the notion of multiple flows of control within a single process. Chapter 11, "Threaded Java", tells you how to write classes that appear to do more than one thing at a time and let you take advantage of powerful multiprocessor hardware.

Big data and data science have become a thing, and Java is right in there. Chapter 12, "Data Science and R" covers some key data science software and the R language. Apache Hadoop, Apache Spark, and much more of the big data infrastructure is written in, and extensible with, Java. Spark is also introduced in this chapter. The R programming

language is popular with data scientists, statisticians, and other scientists. R, like Bell Labs' original S language it was copied from, was implemented in C/C++. At least two reimplementations of R have been written in Java, and Java can also be interfaced directly with the standard R implementation in both directions.

Machine learning (ML) along with generative large language models (LLMs) are key parts of what is termed artificial intelligence (AI). AI has become a hot topic since the release of ChatGPT and its competitors and offspring. Chapter 13, "Machine Learning/Artificial Intelligence", talks about AI in general and then focuses on several Java libraries for applying AI.

Because Java was originally promulgated as the programming language for the internet, it's only fair to spend some time on networking in Java. Chapter 14, "Network Clients", covers the basics of network programming from the client side, focusing on sockets. Today, so many applications need to access a web service, primarily RESTful web services, that this seemed necessary. I'll then move to the server side in Chapter 15, "Server-Side Java", wherein you'll learn some server-side programming techniques.

One simple text-based representation for data interchange is JSON, the JavaScript Object Notation. Chapter 16, "Processing JSON Data", describes the format and some of the many APIs that have emerged to deal with it.

Chapter 17, "Reflection, or "A Class Named Class"", lets you in on such secrets as how to write API cross-reference documents mechanically and how web servers are able to load any old servlet—even if they've never seen that particular class before—and run it.

Sometimes you already have code written and working in another language that can do part of your work for you, or you want to use Java as part of a larger package. Chapter 18, "Using Java with Other Languages", shows you how to run an external program (compiled or script) and also interact directly with native code in C/C++ or other languages.

There isn't room in a book this size for everything I'd like to tell you about Java. The brief Afterword presents some closing thoughts and a link to my online summary of Java APIs that every Java developer should know about.

Finally, Appendix A, "Java Then and Now", gives the storied history of recent Java in a release-by-release timeline, so whatever version of Java you learned, you can jump in here and get up to date quickly. Versions prior to 16 have been relocated to my own website.

So many topics, and so few pages! Many topics do not receive 100% coverage; I've tried to include the most important or most useful parts of each API. To go beyond, check the official *Javadoc* pages for each package; many of these pages have some brief tutorial information on how the package is to be used.

Besides the parts of Java covered in this book, two other platform editions, Java Micro Edition (Java ME) and Java Enterprise Edition (Java EE), have been standardized. Java ME is concerned with small devices such as handhelds, tiny "feature phones" (not really smartphones), fax machines, and the like. At the other end of the size scale—large server machines—there's Eclipse Jakarta EE. This is the follow-on to the former Java EE, which in the last century was briefly known as J2EE. Jakarta EE is concerned with building large, scalable, distributed applications. APIs that are part of Jakarta EE include Servlets, JavaServer Pages, JavaServer Faces, JavaMail, Contexts and Dependency Injection (CDI), JAX-RS for RESTful Web Services, JSON-P and JSON-B for processing JSON-format data, and Transactions. Jakarta EE package names originally began with "javax." because they are not core packages, but have now moved to "jakarta." This book mentions only a few of these. *Jakarta EE Cookbook* by Elder Moraes (Packt Publishing) covers some of the Jakarta EE APIs, as does the older *Java Servlet and JSP Cookbook* by Bruce Perry (O'Reilly).

This book doesn't cover Java ME at all. But speaking of cell phones and mobile devices, you probably know that Android uses Java as its language, as well as Kotlin, which is a simplification and extension of Java. What should be comforting to Java developers is that Android also uses most of the core Java API, except for Swing and AWT, for which it provides Android-specific replacements. The Java developer who wants to learn Android may consider looking at my *Android Cookbook* (O'Reilly), or the book's website.

Java Books

A lot of useful information is packed into this book. However, due to the breadth of topics, it is not possible to give book-length treatment to any one topic. Because of this, the book contains references to many websites and other books. In pointing out these references, I'm hoping to serve my target audience: the person who wants to learn more about Java.

O'Reilly publishes, in my opinion, the best selection of Java books on the market. As the API continues to expand, so does the coverage. Check out the complete list of O'Reilly's collection of Java books; you can buy them at most bookstores, both physical and virtual. You can also read them online through the O'Reilly Online Learning

Platform, a paid subscription service. Though many books are mentioned at appropriate spots in this book, a few deserve special mention. However, to keep this book short, and allow updating, I've created a couple of book lists, one Java specific and one for developers in general (not to mention one for "the educated adult").

Conventions Used in This Book

This book uses the following conventions.

Programming Conventions

I use the following terminology in this book. A *program* means any unit of code that can be run, from a three-line main program to an enterprise component or a full-blown GUI application. An *application* is another name for a program. A *desktop application* (a.k.a. client) interacts with the user. A *server program* deals with a client indirectly, usually via a network connection (usually REST over HTTPS these days).

The examples shown are in two varieties. Those that begin with zero or more import statements, a Javadoc comment, and often a `public class` statement are complete examples. Those that begin with a declaration or executable statement, of course, are excerpts. However, the full versions of all code samples have been compiled and run, and the online source includes the full versions.

Recipes are numbered by chapter and number, so, for example, Recipe 8.1 refers to the first recipe in Chapter 8.

Typesetting Conventions

The following typographic conventions are used in this book:

Italic

Used for commands, filenames, and example URLs. It is also used for emphasis and to define new terms when they first appear in the text.

`*Constant width*`

Used in code examples to show partial or complete Java source code program listings. It is also used for class names, method names, variable names, and other fragments of Java code.

Constant width bold

Used for user input, such as commands that you type on the command line.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

Icons

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element signifies a warning or caution.

SIDEBARS

Paragraphs like this with vertical bars on each side indicate a "sidebar," a discussion that isn't part of any one recipe but may apply to several recipes in the same part of the text.

---

25 This icon (with a particular release number) indicates the minimum Java version for a given explanation or code sample. A *P* after the number indicates that it is a preview feature, thus `--enable-preview` is needed. Similarly, a *C* after the number indicates an incubating feature, requiring `--add-modules jdk.incubator.MMM` (where `MMM` is the incubation module, e.g., `concurrent`, `vector`, etc.).

INCUBATION, PREVIEW, AND PRODUCTION

The Oracle team behind Java are very deep thinkers when it comes to language features, and will never rush a feature into the language, nor let it in without adequate consideration. In order to enable careful phase-in of new functionality, Java puts significant new features into one of two "trial" or "upcoming" states: *preview* and *incubation*. Preview features are nearly final and usually make it into Java after a few releases, whereas incubator features are less complete and more tentative. Brian Goetz explains the difference between preview and incubation:

> *Preview features are really finished but are waiting for a round of feedback, whereas the incubator mechanism has more room to iterate over the API several times to get feedback.*

However, the "string template" preview feature from Java 21 and 22 was withdrawn for rethinking in Java 23, so it presumably wasn't "really finished." Preview features must be enabled with the `--enable-preview` feature, both during compile and execution, and also `--source 21` (or other version number) when compiling (but not when running compiled code). Incubator modules require `--incubation` *MODULENAME* and usually a *module-info* file that depends on the given module.

Another key difference is that preview features provide imports from their expected final package names, so you likely won't have to change your code when the feature comes out of preview. Incubator features, on the other hand, always provide imports from `jdk.incubator.`*MODULENAME*, meaning you definitely will need to change your code (at least the imports) when the feature progresses to preview or to production in a future release.

Using Code Examples

The code samples for this book are in my GitHub repositories. Most are in the *javasrc* repo, but a few are pulled in from another repository, *darwinsys-api*. Details on downloading these are in Recipe 1.3.

Many programs are accompanied by an example showing them in action, run from the command line. These will usually show a prompt ending in either `$` for Unix/Linux/macOS, or `>` for Windows, depending on which computer I happened to be using the day I wrote that example. Text before this prompt character can be ignored; it may be a pathname or a hostname, again, depending on the system.

These examples will usually also show the full package name of the class because Java requires this when starting a compiled program from the command line. Because that will remind you which subdirectory of the source repository contains the source code, I won't point it out explicitly very often.

If you have a technical question or a problem using the code examples, please send email to *support@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Java Cookbook* by Ian F. Darwin (O'Reilly). Copyright 2025 RejmiNet Group, Inc., 978-1-098-16997-8."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning

NOTE

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, conferences, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, please visit *https://oreilly.com*.

Comments and Questions

As mentioned earlier, I have tested all the code on at least one of the reference platforms, and most on several. Still, there may be platform dependencies, or even bugs, in my code or in some important Java implementation. Please report any errors you find, as well as your suggestions for future editions. We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/java-cookbook-5ed*. You can also email *bookquestions@oreilly.com* to comment or ask technical questions about this book. If all else fails, you can contact O'Reilly:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

https://oreilly.com/about/contact.html

For more information about our books, courses, conferences, and news, see our website at *https://www.oreilly.com*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly-media*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

The O'Reilly site lists errata. You'll also find the source code for all the Java code examples to download; *please* don't waste your time typing them again! For specific instructions, see Recipe 1.3.

Acknowledgments

I wrote in the Afterword to the first edition that "writing this book has been a humbling experience." I should add that maintaining it has been humbling, too. While many have

been lavish with their praise—one very kind reviewer called it "arguably the best book ever written on the Java programming language"—I have been humbled by the number of errors and omissions in earlier editions. I have endeavored to correct these.

My life has been touched many times by the flow of the fates bringing me into contact with the right person to show me the right thing at the right time. Steve Munro, who is now an oft-cited Toronto Transportation critic, introduced me to computers when we were in the same class in high school—in particular an IBM 360/30 at the Toronto Board of Education that was bigger than a living room, had 32 or 64*K* (not M or G!) of memory, and had perhaps less computing power than a smartwatch. The late Herb Kugel took me under his wing at the University of Toronto while I was learning about the larger IBM mainframes that came later. Terry Wood and Dennis Smith at the University of Toronto introduced me to mini- and micro-computers before there was an IBM PC. On evenings and weekends, the Toronto Business Club of Toastmasters International and Al Lambert's Canada SCUBA School allowed me to develop my public speaking and teaching abilities. Several people at the University of Toronto, but especially Geoffrey Collyer, taught me the features and benefits of the Unix operating system at a time when I was ready to learn it.

Thanks to the many Learning Tree instructors and students who showed me ways of improving my presentations. I still teach for "The Tree" and recommend their courses for the busy developer who wants to zero in on one topic in detail over a few days.

Closer to this project, Tim O'Reilly believed in my "little Lint book" when it was just a sample chapter from a proposed longer work, enabling my early entry into the rarefied circle of O'Reilly authors. Years later, Mike Loukides encouraged me to keep trying to find a Java book idea that both he and I could work with. And he stuck by me when I kept falling behind the deadlines. Mike also read the entire manuscript and made many sensible comments, some of which brought my flights of fancy down to earth. Jessamyn Read turned many scratchings of dubious legibility into the quality illustrations you see in this book. And many, many other talented people at O'Reilly helped put this book into the form in which you now see it.

The code examples are dynamically included (so updates get done faster) rather than pasted in. My son (and functional programming developer) Benjamin Darwin helped meet the deadline for an earlier edition by converting almost the entire codebase to O'Reilly's newest "include" mechanism and by resolving a couple of other non-Java presentation issues. He also helped make Chapter 9 clearer and more functional.

At O'Reilly

For this fifth edition of the book, my sincere thanks to all the talented people who worked on the manuscript; they're all listed on the publication data page. For the previous (fourth) edition of the book, Suzanne McQuade was the editorial overseer, and Corbin Collins the principal editor. Corbin was especially meticulous in checking the manuscript. Meghan Blanchette, Sarah Schneider, Adam Witwer, Melanie Yarbrough, and the many production people who played a part in getting the third edition ready for you to read. Thanks to Mike Loukides, Deb Cameron, and Marlowe Shaeffer for editorial and production work on the second edition.

Technical Reviewers

For the fifth edition, I received a great deal of quality feedback from George Ball, Jason Clark, Marc Loy, Nighi Nair, and Andrew Slice. George, Jason, and Marc in particular sent lots of specific comments on the manuscript and tried to browbeat me into not letting anything vague make it into the book. Thank you all for the time you took in preparing your feedback. I'm sorry I didn't get 100% of your suggestions in, but each of you has improved the book in many ways, which you may recognize. The errors that remain are on me.

For the fourth edition I was blessed to have two very thorough technical reviewers, Sander Mak and Daniel Hinojosa. Many issues that I hadn't considered during the main revision were called out by these two, leading to extensive rewrites and changes in the last few weeks before the O'Reilly production team took over. Thanks so much to both of you!

My reviewer for the third edition, Alex Stangl, read the third edition manuscript and went far above the call of duty, making innumerable helpful suggestions, even finding typos that had been present in previous editions! Helpful suggestions on particular sections were made by Benjamin Darwin, Mark Finkov, and Igor Savin. For anyone I've forgotten to mention, I thank you all!

Bil Lewis and Mike Slinn made helpful comments on multiple drafts of the first edition. Ron Hitchens and Marc Loy carefully read the entire final draft of the first edition. I am grateful to Mike Loukides for his encouragement and support throughout the process. Editor Sue Miller helped shepherd the manuscript through the somewhat energetic final phases of production. Sarah Slocombe read the XML chapter in its entirety and made many lucid suggestions; unfortunately, time did not permit me to include all of them in the first edition.

Jonathan Knudsen, Andy Oram, and David Flanagan commented on the book's outline when it was little more than a list of chapters and recipes, and they were able to see the kind of book it could become and suggest ways to make it better.

Each of these people made this book better in many ways, particularly by suggesting additional recipes or revising existing ones. Thanks to one and all! The faults that remain are my own.

Readers

My sincere thanks to all the readers who found errata and suggested improvements. Every new edition is better for the efforts of folks like you who take the time and trouble to report that which needs reporting!

Special mention must be made of one of the book's German translators,[2] Gisbert Selke, who read the first edition cover to cover during its translation and clarified my English. Gisbert did it all over again for the second edition and provided many code refactorings, which made this a better book than it would be otherwise. Going beyond the call of duty, Gisbert even contributed one recipe that appeared in several previous editions, and revised some of the other recipes in the same chapter. Thank you, Gisbert!

The second edition also benefited from comments by Jim Burgess, who read large parts of the book. Comments on individual chapters were received from Jonathan Fuerth, the late Kim Fowler, Marc Loy, and Mike McCloskey. My wife, Betty, and my then-teenaged children each proofread several chapters as well.

The following people contributed significant bug reports or suggested improvements: Rex Bosma, Rod Buchanan, John Chamberlain, Keith Goldman, Gilles-Philippe Gregoire, B. S. Hughes, Jeff Johnston, Rob Konigsberg, Tom Murtagh, Jonathan O'Connor, Mark Petrovic, Steve Reisman, Bruce X. Smith, and Patrick Wohlwend.

Etc.

My dear wife, Betty Cerar, still knows more about the caffeinated beverage that I drink while programming than the programming language I use, but her passion for clear expression and correct grammar has benefited so much of my writing during our life together.

No book on Java would be complete without a note of thanks to James Gosling for inventing Java (he also invented the first Unix Emacs, the *sc* spreadsheet, and the NeWS

window system). Thanks also to his employer Sun Microsystems (before they were taken over by Oracle) for releasing not only the Java language but also an incredible array of Java tools and API libraries freely available over the internet.

Willi Powell of Apple Canada provided macOS access for the first edition, back in the early days of macOS.

To each and every one of you, my sincere thanks.

Book Production Software

I used a variety of tools and operating systems in preparing, compiling, and testing this book. The developers of OpenBSD, "the proactively secure Unix-like system," deserve thanks for making a stable and secure Unix clone that is also closer to traditional Unix than other freeware systems. I used the *vi* editor (*vi* on OpenBSD and *vim* on Windows) while inputting the original manuscript in XML, and I used Adobe FrameMaker to format the documents. Frame was a wonderful GUI-based documentation tool that Adobe bought and subsequently "destroyed"; they promised to make it available for macOS but failed to do so for years after macOS 10.0.

The third, fourth, and fifth editions of this *Java Cookbook* were formatted in Asciidoctor and brought to life on the publishing toolchain of O'Reilly's Atlas.

1 For the quirks, see the *Java Puzzlers* books by Joshua Bloch and Neal Gafter (Addison-Wesley).

2 Earlier editions are or have been available in English, German, French, Polish, Russian, Korean, Traditional Chinese, and Simplified Chinese. My thanks to all the translators for their efforts in making the book available to a wider audience.

Previous chapter

‹ Java Cookbook, 5th Edition

Next chapter

1. Getting Started: Compiling and Running Java ›